

ANALOG INSTRUMENTS DETECTION, RECOGNITION, READING & RECORDING

Tareq Aboukhris ¹ ,

¹ Affiliation 1; www.nawa-eng.com.ly

* Correspondence: tareq.alhamaly@yahoo.com; tareq.alhamaly@nawa-eng.com.ly; Tel.: +218917880952

Abstract

The integration of Artificial Intelligence (AI), and particularly computer vision, into industrial operations is rapidly transforming the way facilities are monitored and managed. This study presents a computer vision model designed for the detection, recognition, reading, and recording of analog gauges commonly used in industrial environments, with a specific focus on the oil and gas sector. The model represents the first step toward the broader adoption of AI-driven solutions—especially deep learning-based computer vision models—in outdoor industrial operations. The proposed framework leverages state-of-the-art pretrained models, including YOLOv8 for object detection and recognition, combined with Optical Character Recognition (OCR) and advanced image processing techniques, to achieve accurate and reliable gauge reading. By automating the visual inspection of distributed instrumentation devices, this approach provides an alternative to human operators in tasks that require continuous monitoring of operational conditions. The model demonstrates its potential to enhance operational efficiency, minimize human error, and improve safety in industrial environments where manual monitoring can be labor-intensive and error-prone. As the initial phase of a multi-stage research agenda, this work establishes a foundation for future developments, including noise detection, hazardous fluid leak monitoring, and corrosion detection using deep learning-based non-destructive testing (NDT) methods. Overall, the study highlights how computer vision can serve as a transformative tool in the digitalization of industrial operations.

Keywords: Computer Vision; AI; YOLO

1. Introduction

Today we live in an era where machines can learn on their own, analyze data (regardless of its size) like experts, and even help doctors better diagnose diseases. That's the magic of AI (Artificial Intelligence), and it has already penetrated several industries.

AI can have an immensely positive effect on our professional lives. It promises significant advancements in every aspect of life, such as enhanced efficiency and productivity, which means automating repetitive tasks with AI allows us to say goodbye to tedious jobs and frees us up for more exciting endeavors. In addition to that, it improves decision-making mechanisms. More specifically, Artificial Intelligence can quickly analyze large datasets to uncover hidden patterns we might overlook and make smarter, quicker choices.

One of the most recent and interesting developments in AI is "Computer Vision," which — according to Microsoft's official site — is defined as "a field of computer science that focuses on enabling computers to identify and understand objects and people in images and videos. Like other types of AI, computer vision seeks to perform and

Received:

Revised:

Accepted:

Published:

Citation: Lastname, F.; Lastname, F.; Lastname, F. Title. *Journal Not Specified* **2025**, *1*, 0. <https://doi.org/>

Copyright: © 2025 by the authors. Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

automate tasks that replicate human capabilities.” In industry, this field of technology enables manufacturers to interpret visual data like humans and automate repetitive tasks. Many computer vision applications can cover a wide range of areas, including design, production, quality assurance, packaging, logistics, and dismantling. Moreover, computer vision supports many industries with visual inspection, predictive maintenance, intelligent processes, error detection, automated assembly lines, and robotic collaboration. It also benefits manufacturing processes with higher productivity, greater safety, and fewer human errors.

In the oil and gas sector, computer vision has begun to play a transformative role by enabling real-time monitoring of equipment, detecting leaks, and automating safety inspections. This integration not only reduces operational risks but also enhances efficiency and reliability across manufacturing industries and critical industrial environments.

The proposed work here is just the starting point of a broader multistage solution, which has been divided into several phases that shall be presented in future papers as listed below:-

Table 1. AI-based Outside Operator model implementation Phases Overview

Phase	Description
Phase 1	Analog instruments detection, recognition, reading & recording
Phase 2	Noise detection, recognition and recording in industrial zones
Phase 3	Hazard fluids leak detection model
Phase 4	Corrosion detection and NDT technics implementation using deep learning model

1.1. Location of Study:-

This study took place at the Brega Compressor Station, located in Brega Port on the northeast coast of Libya. The Brega CS is considered one of the most crucial facilities of Sirte Oil Company for the production and processing of oil and gas. It is important because the station serves as the main supply hub for natural gas throughout the Libyan coastal pipeline network to the country’s power plants and other consumers.

1.2. Brega Compressor Station Operation Management System:-

The production/operation manpower distribution and the operation management system in general at Brega CS can be classified into three main categories:

1- Operation Supervisor(s):

The shift supervisor is responsible for managing the operation process during a specific period of time (usually a shift) by overseeing other operational staff, assigning tasks, and making/approving final decisions regarding the operation status.

2- Panel Man (Control Room Operators):

The control room operators are responsible for operating, monitoring, and maintaining all operational parameters within allowable limits by interacting with the facility’s control systems and instrumentation.

3- Outside Operators:

The outside operators are responsible for monitoring operations inside the plant by performing various activities, such as routine visual inspections of equipment, recording operational readings from external indicators (e.g., gauges), observing leaks, and ensuring safety procedures within hazardous area boundaries.

1.3. Issues Regarding the current Operation Management System:-

The third category of the process management chain (outside operators) still relies entirely on human senses to evaluate operational conditions, which are considered a variable factor that differs from site to site and from one operator to another, depending on their experience in the plant.

In other words, the outside operator must go through years of experience and troubleshooting situations to acquire a sufficient understanding of the unit's or plant's operational behavior. On the other hand, during failures in the facility's control system or in cases where the plant must be operated and monitored manually, it becomes critical for all operational staff to handle outside analog devices and remain aware of their safe boundaries individually.

This scenario cannot be reliably achieved due to differences in operators' efficiency; therefore, a robust rapid-learning solution must be introduced to enhance safety and productivity levels in high-risk environments.

1.4. Proposed Deep Learning Solution: :-

The proposed model in this work was designed to overcome the uncertain factors in process management by providing a deep learning model as an alternative monitoring method to observe any process deviation that cannot be noticed by human senses due to lack of operator qualification and experience. The main concept of this model is to capture images of the plant field gauges and feed these images to the deep learning model, not only to identify and record the gauge dial readings but also to determine the process features and their safe boundaries at this specific monitoring point, depending on the gauge tag number and location in the plant P&ID diagrams (piping & instrumentation diagrams).

1.5. Previous Contributions:-

Many algorithms have been developed in this field with various degrees of success to solve all issues related to analog instrumentation reading. However, Edoardo, Oscar, and Jacob [1] applied multi-stage deep learning models for reading gauges in unsaturated environments. They used the YOLO model for gauge detection and image perspective rectification by executing OpenCV libraries (finding circular contours and Hough circle transform). In addition, they performed Hough linear transformation to detect the pointer position and used an OCR engine to identify dial scale readings. It is worthy to mention here that no specific neural network or any alternative deep learning method was designed or applied in this attempt.

Another interesting work by Zhu Li and his counterparts [2] was published in October 2020 under the title: "A High-Robust Automatic Reading Algorithm of Pointer Meters Based on Text Detection." The algorithm implementation stages in this project started with the detection of scale value text based on deep learning (FOTS), then image rectification and center location determination based on text coordinates, followed by a secondary region search method to locate the pointer and scales in the image after polar coordinate conversion, and finally, reading with the distance method within the main scale interval.

Ying Li, Xuemei Li, and Caiming Zhang [3] performed this task by introducing a CNN-based detection and segmentation model in three main components: 1st – meter dial location network, 2nd – reference point segmentation network, 3rd – reading dial number module. In the first model, they used a lightweight version of Darknet-19 (backbone CNN of YOLO) for gauge location detection; secondly, they employed VGG-16 for reference point segmentation; finally, they performed some angle calculations based on center (x, y) coordinates for dial reading extraction.

James Gallagher [4], in his article “Turn Analog Dials into Digital Data with Computer Vision” published on the Roboflow website, proposed a model based on center pointer angle calculation with its corresponding dial reading generated from an object detection model using Roboflow detection tools.

Juan Leon-Alcazar and his colleagues [5] developed another integrated model based on mask segmentation and angular regression model concatenation, then calculated the pointer reading based on the center perspective angle value.

In other similar work, João Peixoto and others [6] provided an entire IoT infrastructure solution in their work: “End-to-End Solution for Analog Gauge Monitoring Using Computer Vision in an IoT Platform” for monitoring operational conditions based on computer vision technology. However, the core model’s architecture for this system is similar to other proposed solutions: starting from capturing the image, then segmenting it using Hough circle transform, and by applying some thresholding and polar transformation, extracting the dial reading based on pointer angle calculation.

Table [2.0] shows all described models and their feature differences:

Table 2. Current methods of analog meter models

Author	Gauge detection	Captured image rectification	Pointer/Dial segmentation	Extracting dial reading
Edoardo et al. [1]	YES	YES	NO	YES
Zhu Li et al. [2]	NO	YES	NO	YES
Ying Li et al. [3]	YES	YES	YES	YES
James Gallagher [4]	NO	NO	NO	YES
Juan Leon-Alcazar et al. [5]	NO	YES	YES	YES
João Peixoto et al. [6]	NO	NO	YES	YES

1.6. paper methodology (CRISP-ML):-

CRISP-ML (Cross-Industry Standard Process for Machine Learning) is one of the standard processes used in industry to overcome business problems with the help of machine learning models. Overall, the CRISP-ML methodology can be divided into the following stages:

- 1- Business & data understanding
- 2- Data preparation
- 3- Model building & tuning
- 4- Evaluation
- 5- Model deployment
- 6- Monitoring & maintenance

The purpose of the first stage (Business and Data Understanding) is to ensure that the project is feasible. That means evaluating whether the task at hand requires machine learning algorithms and what metrics you will use to evaluate the results, whether the data is sufficient and of acceptable quality, and considering the potential economic impact of creating the model and where you will deploy it.

The second step of the CRISP-ML process is to prepare the data for the next stage of modeling. In this phase, tasks such as data selection, data cleaning, feature development, and data transformation are performed. It is desirable to create data transformation pipelines to simplify the task of later model deployment. Data processing and transformation depend on the specific task.

The modeling and model tuning phase is an ML-specific part of the process. The goal is to create one or more machine learning models that will be implemented in the production environment. The choice of algorithms depends on the specific task and technical requirements.

After accomplishing model building and tuning, an evaluation and validation process must be executed in order to determine that the model performs all tasks it has been

designed for. However, the evaluation process must be conducted not only by the designer but also by other users or organizations in multiple situations within its designed purposes.

When the validation step is finalized and the initial weak points are reviewed and repaired, the model should be deployed so that the maximum number of users can utilize it and, therefore, review it. This can be achieved by deploying the model on well-known worldwide web platforms in this field, such as GitHub or any other similar platforms.

The final step of the CRISP-ML standard methodology is to monitor and repair the model's defects or integrity issues during the deployment phase. In other words, when the model is working with real data, the main risk is drift—when the performance of the model drops as it starts working with unknown data. It is very important to monitor the models and identify in time when the model needs to be redesigned.

To summarize, the 1st section of this paper is an introduction and sets the broad outlines of this project. In this section, the historical context of operational process management in industrial environments is presented, along with an introduction to the place where this study was conducted, the system followed for managing operations, the problems associated with this system, the proposed deep learning-based solution, and

2. Materials and Methods

2.1. Introduction:-

The source of the dataset and its preprocessing represent fundamental steps in machine learning, as the quality and reliability of the collected data directly determine the effectiveness of the model. In this study, the dataset originates from field images of analog gauges captured within the industrial facility, serving as the primary source of input information. Since raw data may contain noise, distortions, or irrelevant details, proper preprocessing is essential to refine the dataset and extract the most meaningful features. Therefore, it is critical to carefully preprocess the dataset before feeding it into the model to ensure robust training and accurate predictions.

2.2. Dataset source:-

The dataset used to train our model has been accumulated from Brega Compressor station /Libya for circular pressure and temperature type gauges, the images had been captured by cell phone camera in deferent angles and various conditions, figure [1.0] shows samples of these images:



Figure 1. sample of gauges imagery from Brega compressor station / Libya

The data preparation process -like other computer vision models- has been performed as fixed first step to upcoming stages of model building, however, the reason of manually prepare the dataset is to train the model about the number of classes need to be detect and recognize, also, to give what's so call it "a ground truth" of the real object for the model to compare the training results efficiency.

2.3. Dataset annotation and labeling:-

In short, data annotation is the process of labeling data available in a video, image or text. The data is labeled, so that models can easily comprehend a given data source and recognize certain formats, objects, information, or patterns in the future. The annotation platform used in this work is "Roboflow", it's a widely used tool for data annotation due to its free accessibility for most of platform's features and the simplicity of the labelling process, however, the annotation process in this model was in two stages to generate two weights with specific task each as described below:

Table 3. Overview of the annotation and training process.

Annotation Method	Training Model	Purpose of the Model
Roboflow platform	YOLOv8	To detect gauge's shape or contour
Roboflow platform	YOLOv8	To detect gauge point's Tip and Base

Most common detection CNNs still relay on Manuel annotation to as reference for the model to compare the results with the ground truth. The dataset consists of 200 colored image (3 channel images) with the following distribution:

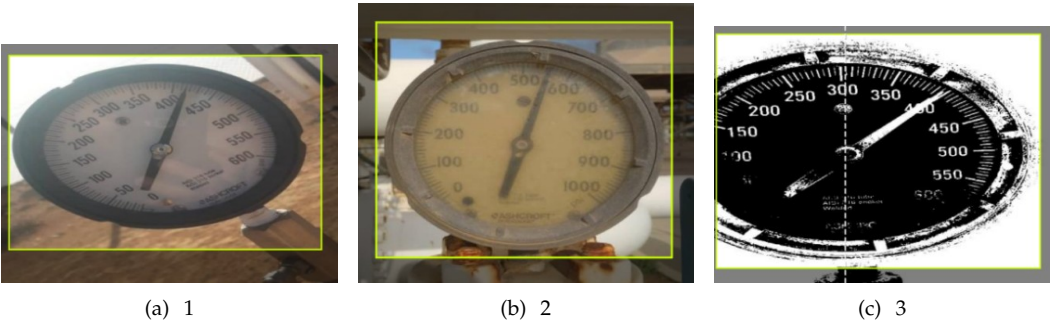


Figure 2. example of the annotated data

Most common detection CNNs still relay on Manuel annotation to as reference for the model to compare the results with the ground truth. The initial dataset of this study consists of 200 colored images (3 channel images) with the following distribution:

- 1. 80% for training dataset (160 images)
- 2. 15% for validation (30 images)
- 3. 5% (10 images) of the dataset for testing purposes

2.4. Dataset Augmentation:-

Data augmentation is the process of artificially generating new data from existing data, primarily to train machine learning models, moreover, machine learning models require large and varied datasets for initial training, but sourcing sufficiently diverse real-world datasets can be challenging because of data silos, regulations, and other limitations. Data augmentation artificially increases the dataset by making small changes to the original data. Generative artificial intelligence (AI) solutions are now being used for high-quality and fast data augmentation in various industries. Deep learning models rely on large volumes of diverse data to develop accurate predictions in various contexts. Data augmentation supplements the creation of data variations that can help a model improve the accuracy of its predictions. Augmented data is vital in training.

The advantages of applying augmentation on the data are the following:

1. Enhanced model performance:

Data augmentation techniques help enrich datasets by creating many variations of existing data. This provides a larger dataset for training and enables a model to encounter more diverse features. The augmented data helps the model better generalize to unseen data and improve its overall performance in real-world environments.

2. Reduced data dependency:

The collection and preparation of large data volumes for training can be costly and time-consuming. Data augmentation techniques increase the effectiveness of smaller datasets, vastly reducing the dependency on large datasets in training environments. You can use smaller datasets to supplement the set with synthetic data points.

3. Mitigate overfitting in training data:

Data augmentation helps prevent overfitting when you’re training ML models. Overfitting is the undesirable ML behavior where a model can accurately provide predictions for

training data but it struggles with new data. If a model trains only with a narrow dataset, it can become overfit and can give predictions related to only that specific data type. In contrast, data augmentation provides a much larger and more comprehensive dataset for model training. It makes training sets appear unique to deep neural networks, preventing them from learning to work with only specific characteristics. In this process of data preparation, 200 annotated/labelled images went through several augmentation processes such as flipping, rotating & blurring to enhance model efficiency to handle most difficult capturing angles, weather & gauge state conditions.

2.5. Model Building & Evaluation:-

As we noticed in section 2.6, most analog meters reader models are follows similar approaches regardless the few differences in implementation procedures, however, the block diagram below shows our analog reader model architecture:

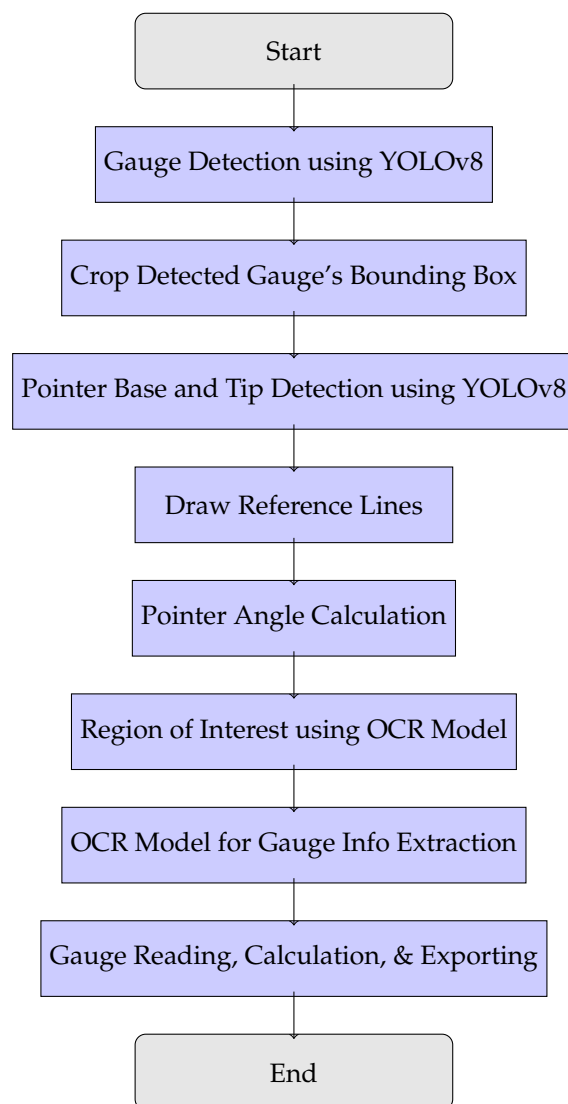


Figure 3. Block diagram of the complete gauge reading workflow.

2.6. Gauge detection using YOLOv8 convolution neural network:-

Although the problem introduced here do not focus on the object detection requirements, still this task playing an important rule to minimize the working domain for the our model by accurately locating the gauge location in the image by using effective deep neural network models, in addition to that, pre-trained convolution neural network-based models

provide precise results in terms of detection tasks, one of the most state-of-art works that has appeared recently is YOLO -you only look once- algorithm, which is designed with the purpose of completing object detection in single forward pass, reducing the computation cost of traditional object multi-step detection processes, due to many advantages other than reducing computation time like its accuracy and simple structure, YOLO and it's variant has been used widely to deal with enormous detection challenges and has achieve excellent results. The selected YOLO iteration used in this project is YOLOv8 which can perform detection, oriented detection, instance segmentation, pose estimation, and classification, YOLOv8 has multiple pre-trained weights* for each task, the one which is chosen here is yolov8x.pt which has the following characteristics: -

Table 4. YOLOv8x Performance and Complexity Metrics.

Weight	Size (px)	mAP	CPU ONNX (ms)	A100 TensorRT (ms)	Params (M)	FLOPs (B)
YOLOv8x.pt	640	53.9	479.1	3.53	68.2	257.8

The training process for YOLOv8 model for gauge detection was executed in google colab platform, the reason for choosing colab due to its provides an available free GPU for machine learning model experiments which overcome the limitation of local machines capabilities. Due to the limited number of the dataset, it was necessary to increase number of training epochs to (100 epochs), with image size 640-pixel, figure [3.0] shows the results of training phase:

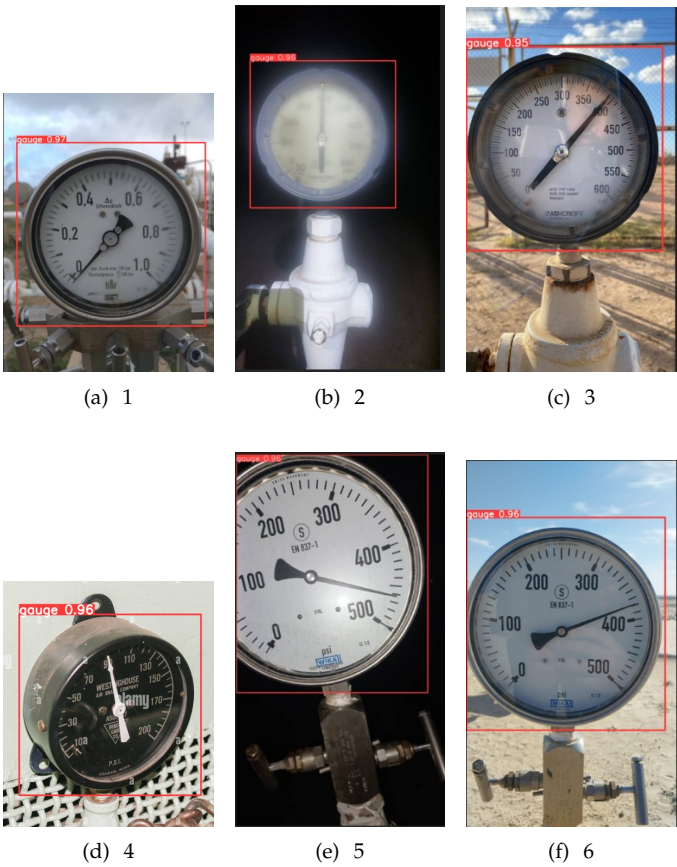


Figure 4. YOLOv8 model training visual result batches

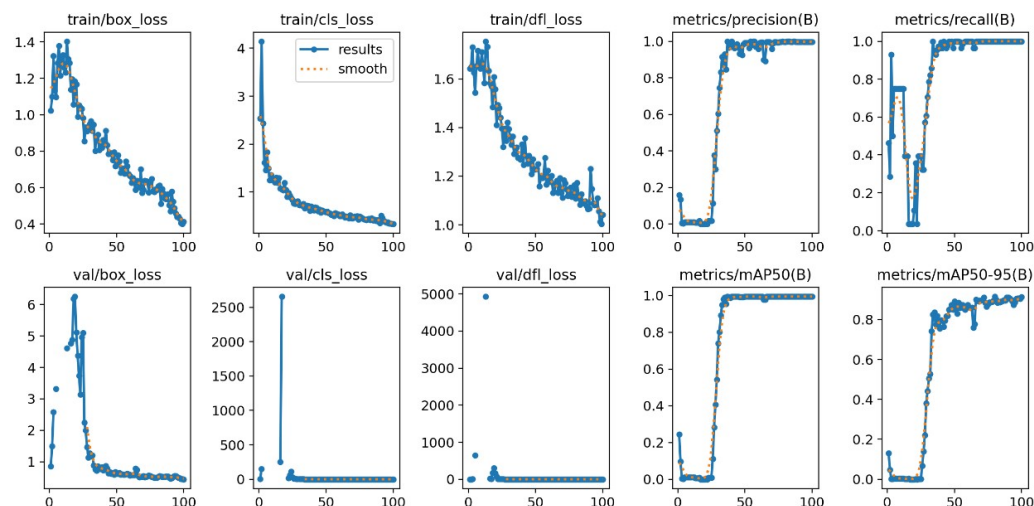


Figure 5. YOLOv8 result figures graphs

Training Evaluation and Metrics

In general, the horizontal axis of the training graph typically represents the number of *epochs*. An epoch is defined as one complete pass of the training dataset through the neural network. Increasing the number of epochs generally leads to better training of the model, as it allows the network to learn the features of the data more thoroughly.

The vertical axis, in a typical YOLO training graph, represents the *loss value*, which is a measure of how well the model is performing during training. Lower loss values indicate better model performance, while higher values suggest poorer performance. The primary goal during training is to minimize the loss value to achieve a well-generalized model.

The main components of the loss function in YOLOv8 are as follows:

- **box_loss:** This is the bounding box regression loss, which measures the error in the predicted bounding box coordinates and dimensions compared to the ground truth. A lower box_loss indicates that the predicted bounding boxes are more accurate.
- **cls_loss:** This is the classification loss, which measures the error in the predicted class probabilities for each object in the image compared to the ground truth labels. A lower cls_loss implies that the model is more accurately predicting the class of the objects.
- **dfl_loss:** This is the loss from the Deformable Convolution Layer, a new addition to the YOLOv8 architecture. This loss measures the error in the deformable convolution layers, which are designed to improve the model's ability to detect objects with varying scales and aspect ratios. A lower dfl_loss indicates that the model is better at handling object deformations and appearance variations.

In addition to the loss components, various metrics are used to evaluate the performance of the YOLOv8 model:

- **metrics/precision:** Represents the accuracy of the detected objects, indicating how many of the predicted detections are correct (true positives over total predicted positives).
- **metrics/recall:** Measures the ability of the model to detect all relevant objects in the images (true positives over total actual positives). High recall indicates the model is finding most of the objects.
- **metrics/mAP50:** Stands for mean Average Precision calculated at an Intersection over Union (IoU) threshold of 0.50. This metric provides a simplified evaluation of the model's detection accuracy for "easier" matches.

- metrics/mAP50-95: This metric represents the average of mAP calculated across multiple IoU thresholds ranging from 0.50 to 0.95 (with a step of 0.05). It offers a comprehensive evaluation of the model's performance across various levels of detection difficulty.

As illustrated, the model's results indicate excellent performance, especially in terms of mean Average Precision (mAP), achieving a value of 0.95. Furthermore, the confusion matrix (based on single-label classification) and an example of model testing using new images are shown in Figure [6.0] below:

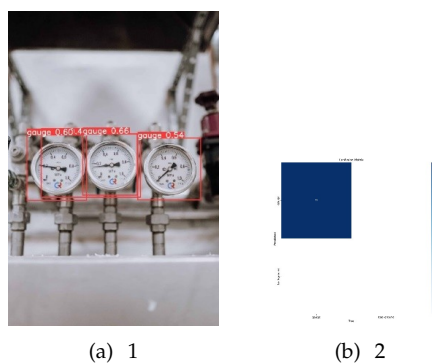


Figure 6. confusion matrix of the model

2.7. Crop Detected Gauge's Bounding Box:-

The next step within the first stage of this model is to crop the result images depending on detected bounding box coordinates, however, the reason of this step is recertificate and minimize background noises which may affect the algorithm detection efficiency. The cropping process was implemented by extracting the top left point and right bottom point vertical and horizontal coordinates of YOLOv8 detected bounding box, and by applying OpenCV library's function "cv2.rectangle()", a rectangle has been drawn between the two points, then by cropping the images to the rectangle exact points coordinates, the results become as figure [7.0] shows below:

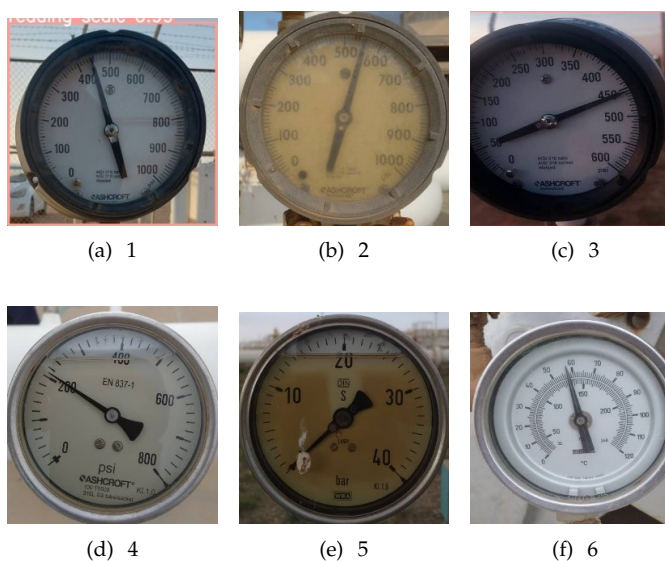


Figure 7. Detected bounding box cropping process

By performing this step, a new dataset has been generated and this dataset is ready now to be fed to train the next YOLOv8 model for detect crucial features within the gauge contour itself.

2.8. Pointer Base and Tip Detection using YOLOv8:-

In order to extract the dial information, and since this model is designed and trained to detect only rotary case gauge instruments, one of the most important steps here – similar other to other algorithms- is to extract the exact location and rotation angle of the pointer (reading needle), in other word, to detect the exact coordinate of the pointer base and tip locations within the image array, to do this, a YOLO model -again- has been trained for desired task by using output (cropped) dataset from previous stage. A total 50 image has been annotated using Roboflow platform which were breakdown to 80% (40 image for training), 15% for validation (8 images), 5% for testing (2 images) and 100 training epochs has been chosen to get an acceptable result as shown below in figure [8.0]: -

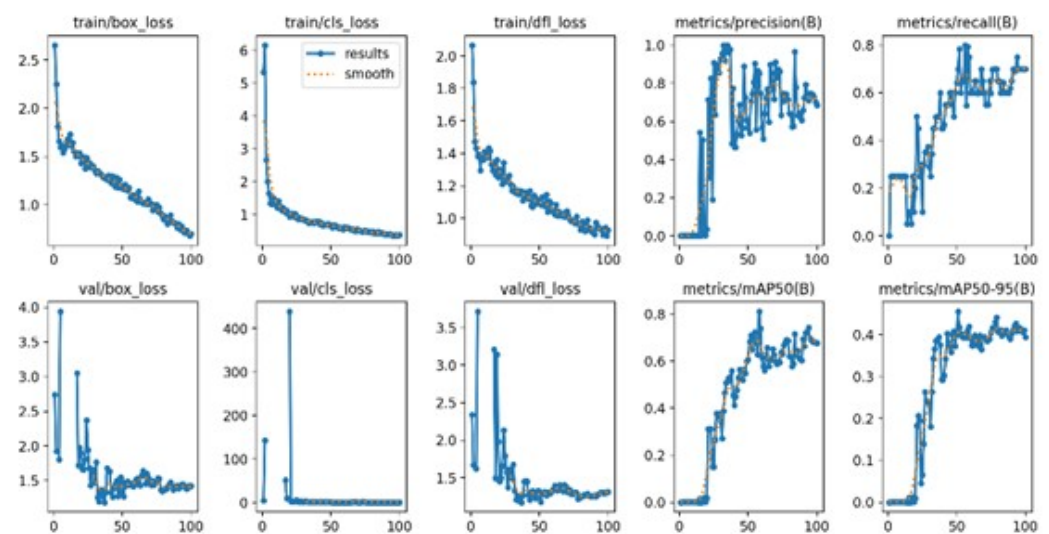
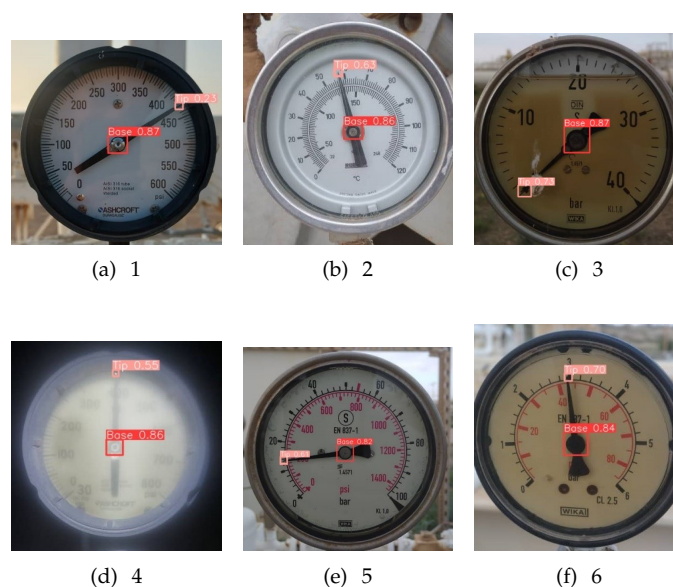


Figure 8. YOLOv8 results for detection pointer's Base and tip

Although the limited number of the dataset images, the YOLOv8 model shows very good results with most cases even with bad condition gauges (for ex: dirty protection glass) with average mAP equal to 0.75, and as long as bigger dataset the training result will improve accordingly

2.9. Draw Reference Lines:-

In order to find the pointer angle with the X axis, reference lines should be drawn one along pointer contour connecting the center of pointer base bounding box and the center of pointer tip bounding box, other is horizontal line to form the required angle with pointer line and the third is connection the tip bounding box with the last point of the horizontal line to form an opposite line for the angle.

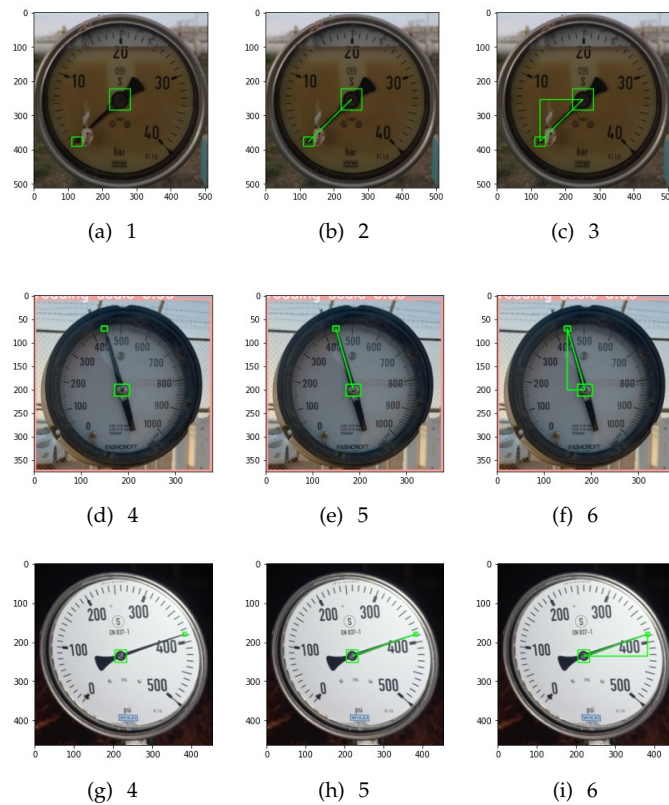


Figure 9. Pointer reference lines

OpenCV library has been utilized to draw both pointer's tip and base bounding boxes, pointer's line, the horizontal and the pointer angle frontier depending upon YOLOv8 detection results as figure [10.0] shows:

```
xywh: tensor([[251.6864, 254.8758, 61.2920, 62.1286],
              [126.7547, 377.2449, 31.1148, 26.7138]])
xywhn: tensor([[0.4964, 0.4988, 0.1209, 0.1216],
               [0.2500, 0.7382, 0.0614, 0.0523]])
xyxy: tensor([[221.0404, 223.8115, 282.3324, 285.9400],
              [111.1973, 363.8880, 142.3121, 390.6019]])
xyxyn: tensor([[0.4360, 0.4380, 0.5569, 0.5596],
               [0.2193, 0.7121, 0.2807, 0.7644]])
```

Figure 10. YOLOv8 bounding boxes coordinates for detection results

The mathematical approach for drawing the mentioned shapes shall be detailed as follows:

- Pointer tip & base bounding boxes:**
The OpenCV function used to draw the pointer’s base box is `cv2.rectangle()`, with input arguments (x_{11}, y_{11}) and (x_{12}, y_{12}) for rectangle top-left point (starting point) and bottom-right point (ending point) respectively.
On the same pattern, the input arguments for pointer tip are (x_{21}, y_{21}) and (x_{22}, y_{22}) as shown in Figure ??.
- Draw reference lines:**
Three reference lines are necessary to calculate the pointer angle with the horizontal axis which can be summarized as follows:
 - Pointer line:**
First line is connecting the center of base bounding box with center of the tip bounding box along pointer contour, which mean the center of the two boxes should be identified and calculated first by using the following formula:

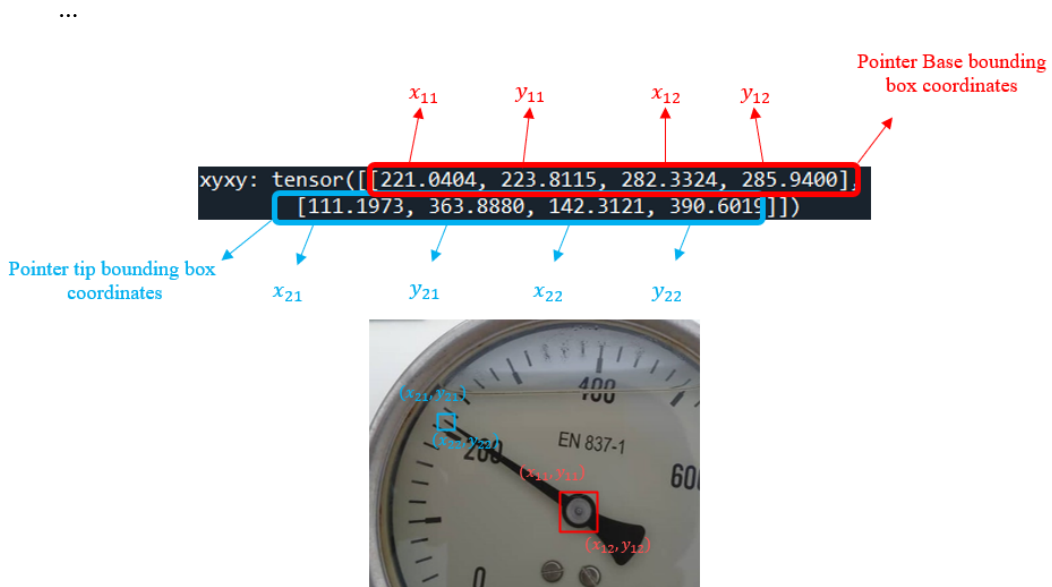


Figure 11. YOLOv8 pointer detection output

Pointer base:	Pointer tip:
X11 = top-left horizontal coordinate	X21 = top-left horizontal coordinate
Y11 = top-left vertical coordinate	Y21 = top-left vertical coordinate
X12 = Bottom-right horizontal coordinate	X22 = Bottom-right horizontal coordinate
Y12 = Bottom-right vertical coordinate	Y22 = Bottom-right vertical coordinate

Figure 12. Pointer’s base and tip bounding boxes coordinates

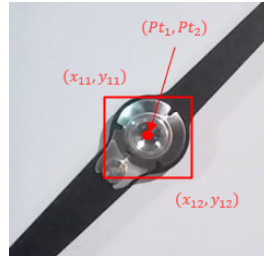
Table 5.

Pt1 = pointer's base bounding box center point horizontal coordinate

Pt2 = pointer's base bounding box center point vertical coordinate

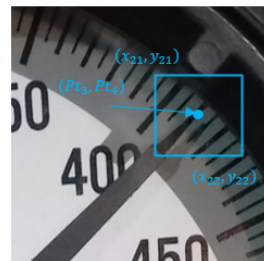
Pt3 = pointer's tip bounding box center point horizontal coordinate

Pt4 = pointer's tip bounding box center point vertical coordinate



$$Pt_1 = \frac{x_{11} + x_{12}}{2} \quad (1)$$

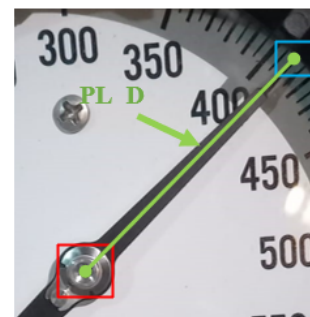
$$Pt_2 = \frac{y_{11} + y_{12}}{2} \quad (2)$$



$$Pt_3 = \frac{x_{21} + x_{22}}{2} \quad (3)$$

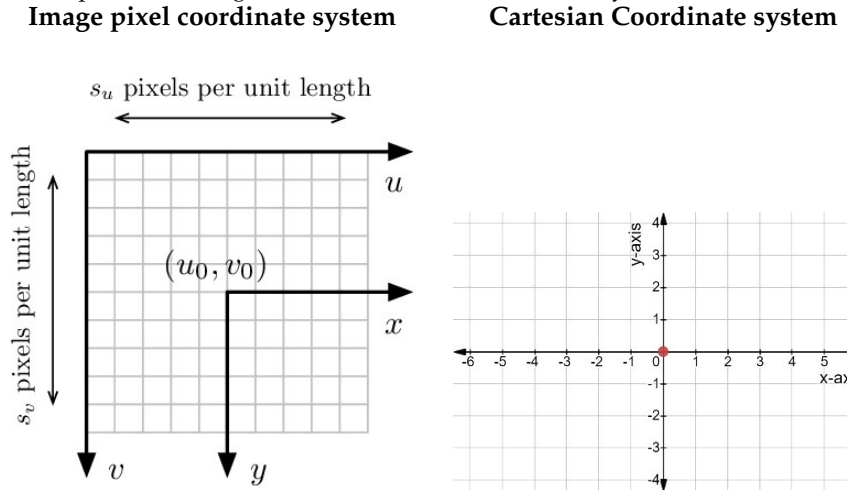
$$Pt_4 = \frac{y_{21} + y_{22}}{2} \quad (4)$$

The second step is to draw a line that connect the two centers using "cv2.line()" function and calculating the line length using the following equation: 372
373

Table 6. Pointer line length calculation

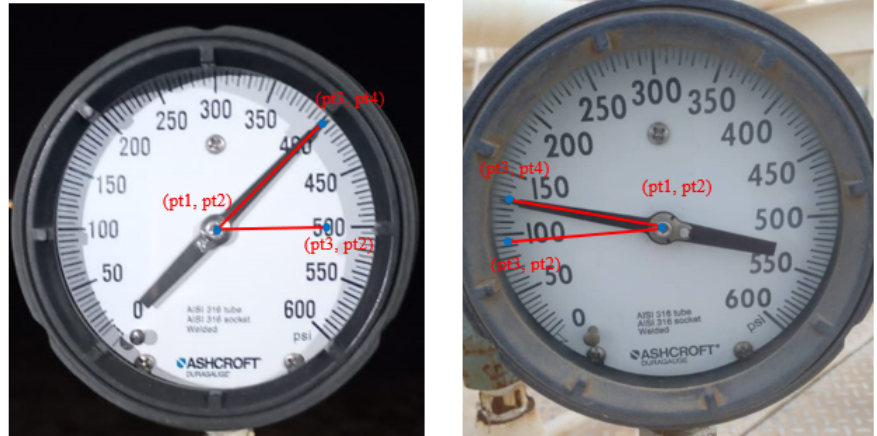
$$PLD = \left| \sqrt{(pt_3 - pt_1)^2 + (pt_4 - pt_2)^2} \right| \quad (5)$$

Equation (5.0) calculate the line length between two points in Cartesian coordinate system, but however, due to the difference between cartesian coordinates system with image pixel system in terms of the location of the origin point which is located in the center of the Cartesian system and in the top left of the image pixel system as figure [13.0] shows, therefor, the absolute value should be consider in the equation to avoid the negative results: 374
375
376
377
378

Figure 13. Comparison of Image Pixel and Cartesian Coordinate Systems

1. **Horizontal Line:**

The pointer rotating angle is located between pointer line and another horizontal line goes from the center point of the pointer base bounding box to same direction of the pointer movement in parallel to the x axis:

Figure 14. Drawing of the horizontal line

As illustrated in Figure ??, the horizontal line starts from point (pt_1, pt_2) and ends at point (pt_3, pt_2) . The length of this line is calculated using Equation (5.0):

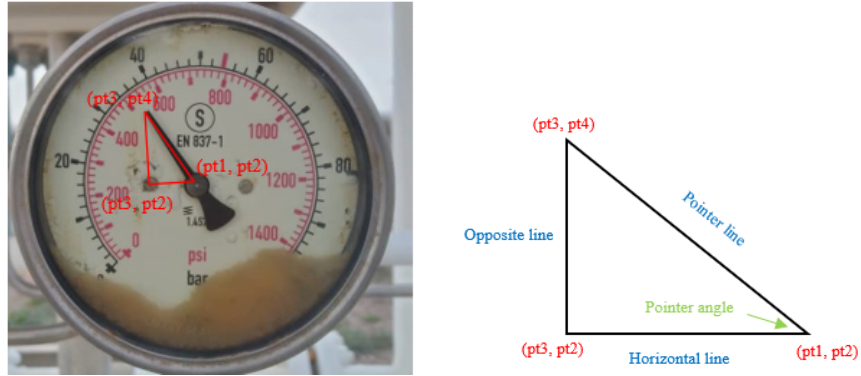
$$HL = \left| \sqrt{(pt_1 - pt_3) + (pt_2 - pt_2)} \right|$$

Since $pt_2 - pt_2 = 0$, the equation simplifies to:

$$HL = \left| \sqrt{pt_1 - pt_3} \right| \quad (6.0)$$

2. **Opposite Line:**

The third and final line to form the right-angled triangle is the opposite line to the pointer angle. This line connects the points (pt_3, pt_2) and (pt_3, pt_4) , as shown in Figure [14.0].

Figure 15. Drawing of the pointer's angle

Similar to the pointer and horizontal line length calculations, the opposite line will be equal to:

$$OL = \left| \sqrt{(pt_3 - pt_3) + (pt_2 - pt_4)} \right|$$

Since $pt_3 - pt_3 = 0$, the equation becomes:

$$OL = \left| \sqrt{pt_2 - pt_4} \right| \quad (7.0)$$

2.10. Pointer Angle Calculation:-

Now, after successfully calculating the lengths of all three lines forming the right-angled triangle, it becomes straightforward to determine the pointer angle by applying a basic trigonometric equation, where the angle can be obtained using the relationship between the opposite and horizontal lines.

$$\text{Angle} = \sin^{-1} \left(\frac{\text{opposite}}{\text{Hypotenuse}} \right), \quad \text{Pointer angle} = \sin^{-1} \left(\frac{\text{opposite line}}{\text{pointer line}} \right) \quad (8.0)$$

The angle result from equation (8.0) will be measured in radians, therefore, a conversion formula must be applied to convert the result to degrees:

$$\text{Pointer angle } (^{\circ}) = \text{Pointer angle (radians)} \times \frac{180^{\circ}}{\pi} \quad (9.0)$$

This formula Equation [9.0] was implemented in Python by importing the math library and applying the `round(math.degrees(A))` function.

```
abs(round(math.degrees(A)))
```

Figure 16. Angle conversion from radians to degrees

the pointer angle has been calculated with respect to the pointer reference horizontal line, however, this calculation does not consider the gauge dial scale starting point (zero reading point) which is located usually in the 1st quarter of the image array as shown in figure [17.0]:

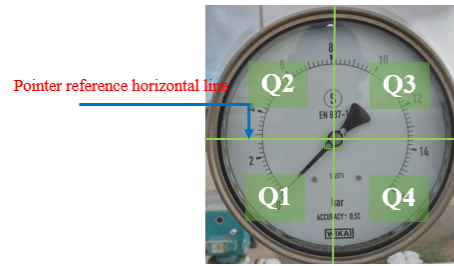


Figure 17. Four quarter of a gauge image

Q1 = first quarter of an image array

Q2= second quarter of an image array

Q3 = third quarter of an image array

Q4 = fourth quarter of an image array

To solve this issue, a correction and conditional functions must be fed to the model to calculate the angle accurately with respect to the dial scale starting point, but before executing this step, it's important here to teach the model where the pointer's tip located in other word in which quarter pointed specifically, to execute this, a creative method has been adopting to allow the model to recognize the pointer tip location depending upon the pointer base bounding box coordinate points (pt1, pt2) and the pointer tip bounding box coordinate points (pt3, pt4). To dive into more details, if we pay a close attention, we notice that in Q1, the value of pt1 is bigger than the value of pt3 with respect to point (0,0) at the top-left of the image coordinate system, correspond to that, at the same quarter (Q1) the value of pt4 is bigger than pt2, in addition, at Q2 the value of pt1 is again bigger than pt2 but the difference here the value of pt2 is bigger when we compare it pt4. Moving toward Q3, in this quarter pt3 will be bigger than pt1 and pt2 is bigger than pt4, while in Q4 pt3 is bigger than pt1 and pt4 is bigger than pt2, figure [00] shows the four-image array quarter with its coordinate point values:

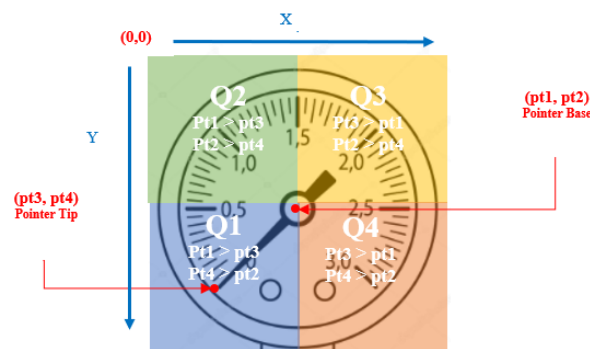


Figure 18. Four-image array quarters with its coordinate point values

After determining the pointer tip location, it is now time to perform the correction and conditional functions to the pointer angle value. The following equations (10.0–13.0) describe these adjustments:

$$\text{If the pointer tip is at Q1: Pointer angle} = \text{Pointer angle} - 45^\circ \quad (10.0)$$

$$\text{If the pointer tip is at Q2: Pointer angle} = \text{Pointer angle} + 45^\circ \quad (11.0)$$

$$\text{If the pointer tip is at Q3: Pointer angle} = (90^\circ - \text{Pointer angle}) + 135^\circ \quad (12.0)$$

$$\text{If the pointer tip is at Q4: Pointer angle} = \text{Pointer angle} + 225^\circ \quad (13.0)$$

2.11. Region of Interest for OCR Model:-

The implementation of a region of interest (ROI) is a crucial task for a wide range of image preprocessing operations. The usage of the ROI process allows the model to focus only on certain boundaries of input dataset batches in each single image, which leads to more accurate results, faster running time, and lower computational costs. Moreover, the ROI region is usually defined based on automatic boundaries or manually drawn geometric shapes. By excluding other regions through image filtering, the model can work only within the specified area.

The ROI method is a fundamental requirement for the upcoming step in this model, which is the OCR process (Optical Character Recognition) – discussed in Section 4.7 – to extract very important information from the gauge background scale. Two ROIs have been applied in two main regions of the gauge's background: the first region corresponds to Q1, the image quarter described above, while the second region is located in the middle-bottom between Q1 and Q4. The reasons for choosing these areas are the following:

Table 7. The two regions of interest applied in the model

Region of Interest Location	Reason for Choosing This Region
Certain portion of Q1 quarter	To extract the dial increasing numerical value (gradians value)
Middle-bottom area between Q1 and Q4	To identify and extract the unit of measure used in the instrument

The execution process of this step starts by converting the 3-channel colored images into grayscale images using the OpenCV library function `cv2.cvtColor()`. Afterward, reference points are identified to draw the geometric boundaries of the ROI, as described below:

The execution process of this step starts by converting the 3-channel colored images into grayscale images using the OpenCV library function `cv2.cvtColor()`. Afterward, reference points are identified to draw the geometric boundaries of the ROI, as described below:

(a) Unit of Measure ROI Reference Points:

The unit of measurement ROI region starts from the center point of the pointer's base bounding box (pt_1, pt_2). From this point, the ROI extends to a second point corresponding to 25% of the image's horizontal dimension as the x value, and the maximum vertical dimension of the image (bottom edge) as the y value. Next, the ROI extends horizontally to a third point located at 75% of the image's horizontal dimension as the x value, while maintaining the same y coordinate as the second point. Figure [00] illustrates this configuration.

The execution process of this step starts by converting the 3-channel colored images into grayscale images using the OpenCV library function `cv2.cvtColor()`. Afterward, reference points are identified to draw the geometric boundaries of the ROI, as described below:

- Unit of Measure ROI Reference Points:** The unit of measurement ROI region starts from the center point of the pointer's base bounding box (pt_1, pt_2). From this point, the ROI extends to a second point corresponding to 25% of the image's horizontal dimension as the x value, and the maximum vertical dimension of the image (bottom edge) as the y value. Next, the ROI extends horizontally to a third point located at 75% of the image's horizontal dimension as the x value, while maintaining the same y coordinate as the second point. Figure [19.0] illustrates this configuration:

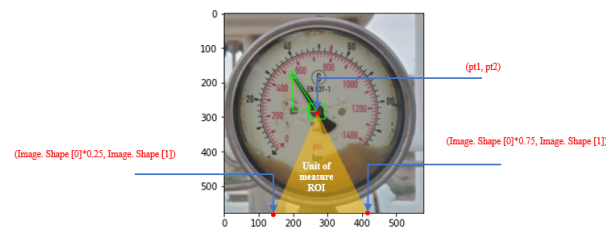


Figure 19. Reference points for unit of measure ROI

Gradians Value ROI:

The reference points for this region start from the center point of the pointer's base bounding box (pt_1, pt_2) , extending horizontally to the left side toward the point $(0, pt_2)$, where the x value of the image shape equals zero. From this point, the boundary extends downward along the $x = 0$ axis toward the point defined as $(0, \text{Image.Shape}[1] + \text{Image.Shape}[0] \times 0.25)$, as illustrated in Figure [20.0]:-

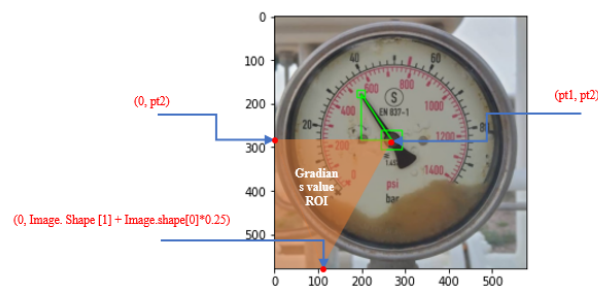


Figure 20. Reference points for gradians value ROI

After determining the reference points for both ROIs, the second step is to convert the input image entirely into a blank image. More precisely, this means converting every pixel value within the image array to zero by applying the `np.zeros_like()` function from the NumPy library on the grayscale image obtained from the previous step. The next step is to identify the ROI boundaries by applying the `np.array()` function, where the input arguments correspond to the reference points specified earlier. The purpose of this step is to provide the model with the exact areas that should be excluded from the blanking process.

The two final steps start with using `cv2.fillPoly()` from the OpenCV library to draw filled shapes over specific regions of the image, in this case, the pre-determined arrays representing both ROIs. Finally, the ROI array formed by the `fillPoly()` function is merged with the blank array by applying the `cv2.bitwise_and()` function. Figure [00] illustrates the final results of performing both ROIs.

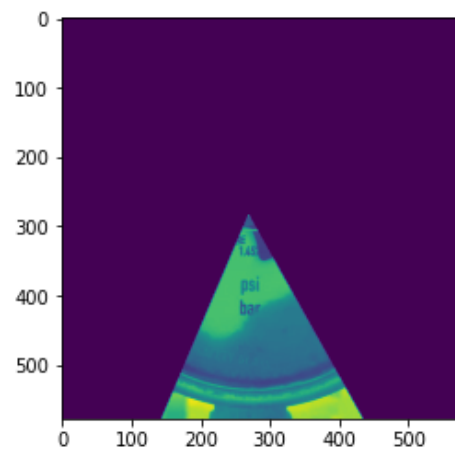


Figure 21. Unit of Measure ROI

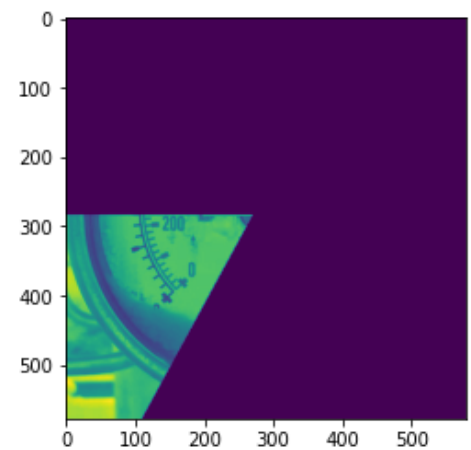


Figure 22. Gradient value ROI

2.12. OCR Model for Gauge Information Extraction:-

Optical Character Recognition (OCR) is the use of technology to distinguish printed or handwritten text characters inside digital images of physical documents, such as scanned paper documents. The basic process of OCR involves examining the text of a document and translating the characters into code that can be used for data processing. OCR is sometimes also referred to as *text recognition*.

OCR systems are composed of both hardware and software. Hardware, such as optical scanners or specialized circuit boards, is used to capture or read text, while software typically handles advanced processing. Modern OCR software often integrates artificial intelligence (AI) techniques to implement intelligent character recognition (ICR), enabling capabilities such as identifying languages or handwriting styles.

The most common use of OCR technology is to convert hard-copy documents (e.g., legal or historical archives) into digital soft-copy formats such as PDFs. Once digitized, these documents can be edited, formatted, or searched as if they were originally created using a word processor.

A wide variety of OCR models and frameworks are available today, each with its own advantages and application domains. Among the most commonly used are:

- (a) Tesseract (locally via PyTesseract)
- (b) EasyOCR (local)
- (c) Surya (local)
- (d) DocTR (via Roboflow Hosted API)
- (e) OpenAI GPT-4 with Vision
- (f) Google Gemini Pro 1.0
- (g) Google Gemini Pro 1.5
- (h) Anthropic Claude 3 Opus
- (i) Others ...

In this work, the model chosen is **EasyOCR**, a Python-based OCR framework that is both flexible and straightforward to use. OCR technology is useful for various tasks, including data entry automation and image analysis, by enabling computers to identify and extract text from images or scanned documents.

EasyOCR stands out for its user-friendliness, even for developers without prior experience in OCR or computer vision. It provides multiple language support, pre-trained text detection and recognition models, and emphasizes both speed and efficiency in text extraction. Its robustness in handling diverse fonts and text layouts, combined with a strong focus on accuracy, makes it a reliable option for Python developers.

EasyOCR greatly simplifies the process of extracting textual information from images for integration into broader projects.

The framework of EasyOCR, as adopted in this work, is illustrated in Figure[23.0].

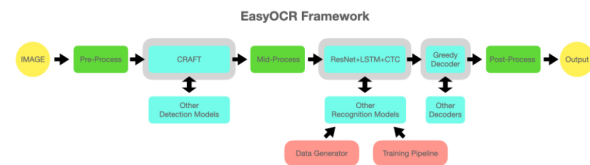


Figure 23. EasyOCR framework

The EasyOCR engine has been applied twice in this model for both ROIs, the first ROI shows the following results:

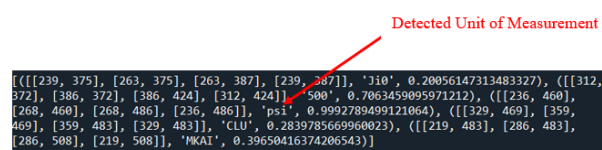


Figure 24. OCR results on unit of measure ROI

The reason for detecting and extracting the unit of measurement is that the model has been designed to read different gauge parameter types, each associated with different units of measurement. However, during experimental applications of EasyOCR under various conditions (e.g., different fonts of unit representation or variations in gauge glass clarity), the detection results appeared in different forms, such as “pSI”, “Psi”, “barG”, and others. These variations may lead to confusion in the reading process. Therefore, a conditional function has been implemented to normalize all potential detection outcomes and standardize them into fixed outputs: “psig”, “Barg”, “°C”, and “°F”.

The second implementation of EasyOCR is the detection and extraction of the scale gradian major value. This step is crucial for two reasons:

- To calculate the gauge reading based on the pointer angle value.
- To determine the appropriate reading equation, depending on the scale gradian value.

The scale gradian value can vary significantly, ranging from 1 to 200, depending on the measured parameter and the corresponding unit of measurement:

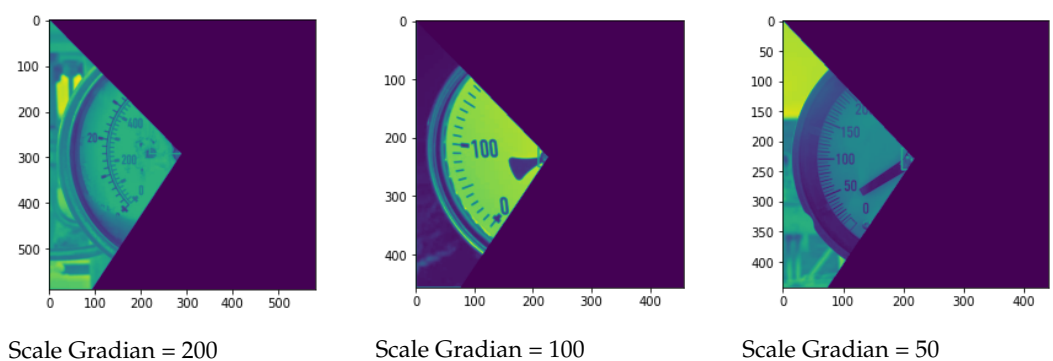
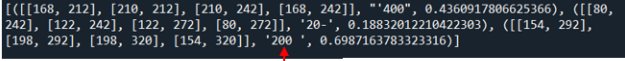


Figure 25. Comparison of gauges scale in gradians

EasyOCR results for this ROI shows in figure [26]:



[[[168, 212], [210, 212], [210, 242], [168, 242]], "'400", 0.4360917806625366), ([[80, 242], [122, 242], [122, 272], [80, 272]], '20-', 0.18832012210422303), ([[154, 292], [198, 292], [198, 320], [154, 320]], '200 ', 0.6987163783323316)]

Detected Scale Gradian Value = 200

Figure 26. OCR results on unit of measure ROI

Although issues regarding text accuracy which was solved by applying conditional functions same to previous case, but overall, the results were quite accurate despite the image condition.

2.13. Gauge Reading Calculation and Data Extraction on Excel File:-

The final and most important step of the model development is to calculate the gauge reading numerical value based on the pointer's angle. To achieve this, several assumptions must be considered as initial conditions for the mathematical approach presented here. The main assumption is that the relationship between the angle value and the gauge reading value follows a **linear relation**.

A linear relationship (or linear association) is a statistical concept describing a straight-line relationship between two variables. It can be expressed either graphically or mathematically as:

$$y = mx + b \quad (1)$$

where m is the slope and b is the y -intercept.

In the general form of the linear equation (1), x and y are two variables related by the parameters m and b . Graphically, the equation $y = mx + b$ is plotted in the x - y plane as a straight line with slope m and intercept b , which represents the value of y when $x = 0$.

There are three necessary criteria for an equation to be classified as linear:

- It cannot contain more than two variables.
- All variables must be raised only to the first power.
- Its graph must be represented as a straight line.

In this work, equation (1) is reformulated by substituting x and y with the pointer angle and gauge reading, respectively, yielding equation (2):

$$\text{Reading} = (m \times \text{Pointer Angle Value}) + b \quad (2)$$

To determine the values of m and b , and since the model is designed to read multiple gauge types, the first step is to identify the scale gradian numerical value and therefore the corresponding equation to be applied. For this purpose, a conditional function has been implemented to select the appropriate linear relation depending on the results obtained from the EasyOCR model for the scale gradian ROI. Figure ?? illustrates the criteria for linear equation selection along with the model output.

Table 8. Applied linear equations for different scale gradian values

Scale Gradian Value	Applied Equation
20	Reading = (0.5348 × Pointer Angle) + 0.3395
50	Reading = (0.5133 × Pointer Angle) + 2.6201
100	Reading = (3.6793 × Pointer Angle) + 5.5341
	Reading = (2.9699 × Pointer Angle) + 5.5639
	Reading = (1.9788 × Pointer Angle) + 6.5448
200	Reading = (5.1052 × Pointer Angle) + 3.9402

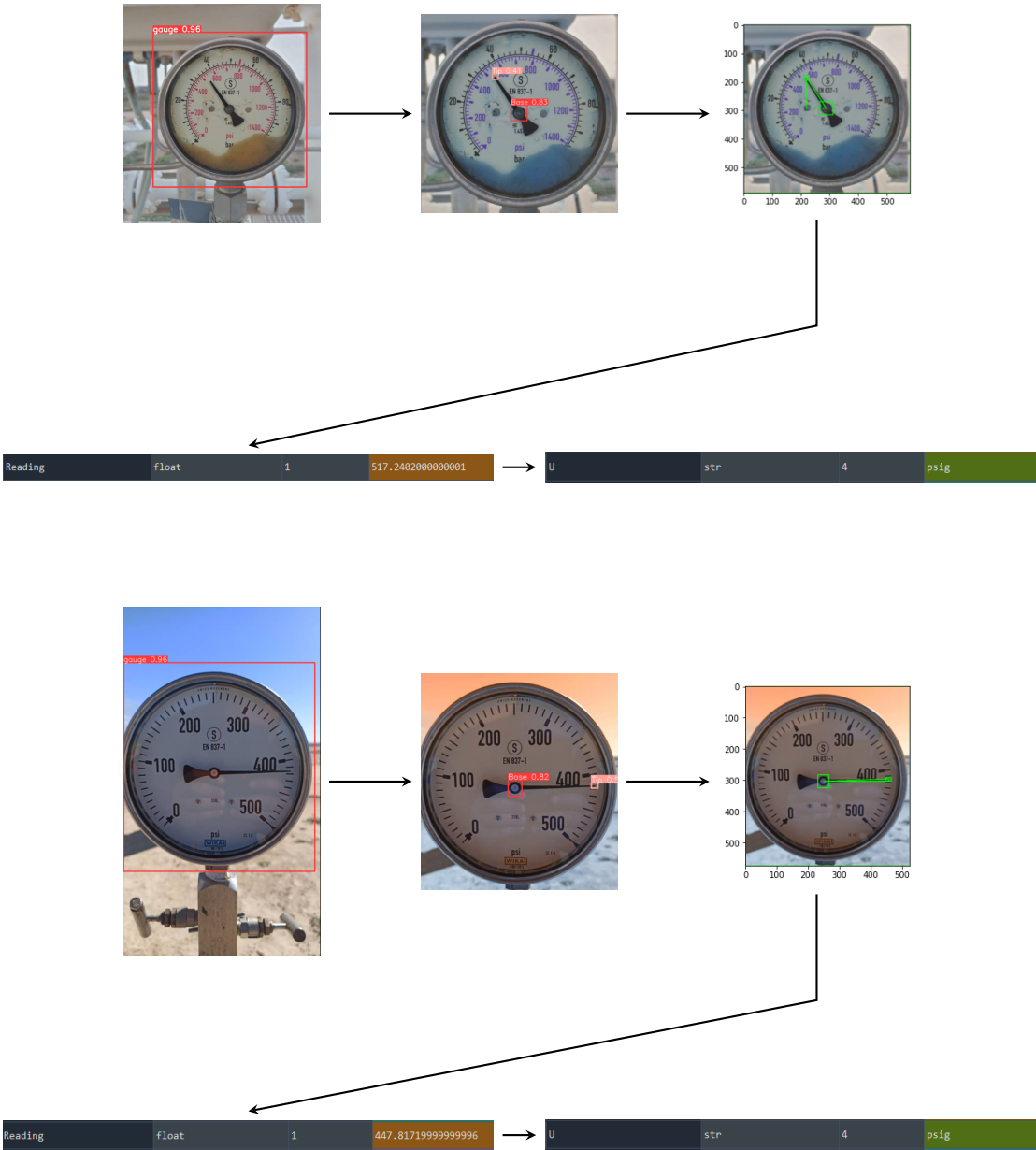


Figure 27. Model final results

Last step in this stage of the model is to save the output data on excel worksheet for recording purposes, these readings is quietly important to archive equipment and units in general behavior and to review their operational historical logs.

By using python library "XlsxWriter" it's easy to write and edit on Microsoft excel files and format the file for any require data type, it also can be used to write text, numbers, formulas and hyperlinks to multiple worksheets and it supports features such as formatting and many more. Figure [27.0] shows example of outside reading sheet:

Company Logo	Company Name	XXX - XXXXXX Reading Sheet												Page 1 of 1
		Issued Date	Edit Date	Form No.										
Unit (XXXX) Reading														
Date:	XX/XX/20XX													
Time	08:00	10:00	12:00	14:00	16:00	18:00	20:00	22:00	00:00	02:00	04:00	06:00	Unit of Measure	Remarks
Fuel Gas Skid Inlet Pressure	517.2												psig	Normal Operation
LV-2 Downstream Pressure	447.81												psig	HHI alarm or the gauge is NOT suitable for the measuring range
							</							

Figure 28. Outside reading sheet

2.14. Conclusion:-

In this section we traveled across model construction and we presented all steps for building the model starting from gauge detection using YOLO, image cropping, pointer’s base and tip detection, drawing references lines, pointer angle calculations, ROI for OCR model, applying OCR for gauge information extraction and finally reading calculation and writing data on excel file for recording purposes.

3. Results

In this section we evaluate and assign the performance of the model which is responsible for accuracy and reliability, this step is essential to ensure that the model function as intended and produce dependable results when deployed in real-world applications. Testing of the model involves various types of assessments and evaluations to verify the quality and effectiveness of these models. These assessments aim to identify and mitigate issues, errors, or biases ensuring that they meet their intended objectives.

3.1. Model deployment and testing:-

As CRISP-ML methodology require, in this section we deploy and test the model in real/different cases and conditions to stand on its effectiveness in such scenarios, however, the model has been tested in newly dataset that has been NOT recognize by the model before, table [5.0] shows the results and performance of the model:

Table 9. Detection results for Images 1–20

Image Name	Gauge	Pointer Base	Pointer Tip	Reading	Unit
1.jpg	YES	YES	NO	NO	NO
2.jpg	YES	YES	NO	NO	NO
3.jpg	YES	YES	NO	NO	NO
4.jpg	YES	YES	YES	YES	NO
5.jpg	YES	YES	NO	NO	NO
6.jpg	YES	YES	YES	YES	YES
7.jpg	YES	YES	NO	NO	NO
8.jpg	YES	YES	NO	NO	NO
9.jpg	YES	YES	YES	YES	YES
10.jpg	YES	YES	NO	NO	NO
11.jpg	YES	YES	NO	NO	NO
12.jpg	YES	YES	NO	NO	NO
13.jpg	YES	YES	NO	NO	NO
14.jpg	YES	YES	YES	YES	YES
15.jpg	YES	YES	YES	YES	YES
16.jpg	YES	YES	NO	NO	NO
17.jpg	YES	YES	YES	YES	YES
18.jpg	YES	YES	YES	YES	NO
19.jpg	YES	YES	NO	NO	NO
20.jpg	YES	YES	YES	YES	YES

Table 10. Detection results for Images 21–40

Image Name	Gauge	Pointer Base	Pointer Tip	Reading	Unit
21.jpg	YES	YES	NO	NO	NO
22.jpg	YES	YES	NO	NO	NO
23.jpg	YES	YES	YES	YES	YES
24.jpg	YES	YES	NO	NO	NO
25.jpg	YES	YES	NO	NO	NO
26.jpg	YES	YES	NO	NO	NO
27.jpg	YES	YES	YES	YES	YES
28.jpg	YES	YES	NO	NO	NO
29.jpg	YES	YES	NO	NO	NO
30.jpg	YES	YES	NO	NO	NO
31.jpg	YES	YES	NO	NO	NO
32.jpg	YES	YES	YES	YES	YES
33.jpg	YES	YES	NO	NO	NO
34.jpg	YES	YES	YES	YES	YES
35.jpg	YES	YES	YES	YES	YES
36.jpg	YES	YES	YES	YES	YES
37.jpg	YES	YES	NO	NO	NO
38.jpg	YES	YES	YES	YES	YES
39.jpg	YES	YES	YES	YES	YES
40.jpg	YES	YES	YES	YES	YES

Table 11. Detection results for Images 41–60

Image Name	Gauge	Pointer Base	Pointer Tip	Reading	Unit
41.jpg	YES	YES	YES	YES	YES
42.jpg	YES	YES	NO	NO	NO
43.jpg	YES	YES	NO	NO	NO
44.jpg	YES	YES	NO	NO	NO
45.jpg	YES	YES	NO	NO	NO
46.jpg	YES	YES	NO	NO	NO
47.jpg	YES	YES	NO	NO	NO
48.jpg	YES	YES	NO	NO	NO
49.jpg	YES	YES	YES	YES	YES
50.jpg	YES	YES	NO	NO	NO
51.jpg	YES	YES	NO	NO	NO
52.jpg	YES	YES	NO	NO	NO
53.jpg	YES	YES	YES	YES	YES
54.jpg	YES	YES	YES	YES	YES
55.jpg	YES	YES	YES	YES	YES
56.jpg	YES	YES	YES	YES	YES
57.jpg	YES	YES	NO	NO	NO
58.jpg	YES	YES	YES	YES	YES
59.jpg	YES	YES	YES	YES	YES
60.jpg	YES	YES	YES	YES	YES

Table 12. Detection results for Images 61–80

Image Name	Gauge	Pointer Base	Pointer Tip	Reading	Unit
61.jpg	YES	YES	YES	YES	YES
62.jpg	YES	YES	YES	YES	YES
63.jpg	YES	YES	NO	NO	NO
64.jpg	YES	YES	NO	NO	NO
65.jpg	YES	YES	NO	NO	NO
66.jpg	YES	YES	NO	NO	NO
67.jpg	YES	YES	NO	NO	NO
68.jpg	YES	YES	YES	YES	YES
69.jpg	YES	YES	NO	NO	NO
70.jpg	YES	YES	NO	NO	NO
71.jpg	YES	YES	NO	NO	NO
72.jpg	YES	YES	NO	NO	NO
73.jpg	YES	YES	YES	YES	YES
74.jpg	YES	YES	NO	NO	NO
75.jpg	YES	YES	YES	YES	YES
76.jpg	YES	YES	YES	YES	YES
77.jpg	YES	YES	YES	YES	YES
78.jpg	YES	YES	NO	NO	NO
79.jpg	YES	YES	NO	NO	NO
80.jpg	YES	YES	YES	YES	YES

As shown in Table[9,10,11,12], the model successfully detected the gauge shape and pointer base with a 100% success rate. The detection rates for the remaining tasks are summarized below:

597

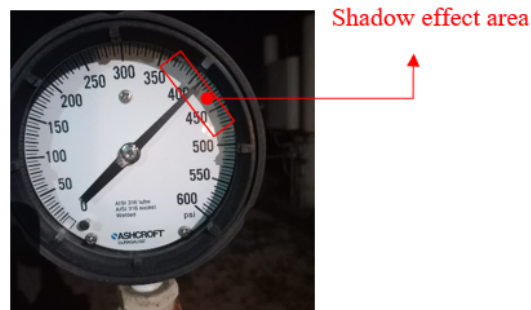
598

599

Table 13. Model evaluation features

Model Evaluation Feature	Formula Used for Calculating the Ratio	Result
Pointer Tip Detection	$R_{\text{tip}} = \frac{D_t}{T} \times 100$ Where: R_{tip} = detection ratio for pointer tip, D_t = Number of detected tips, T = total dataset size	43.75%
Reading Calculation	$R_{\text{reading}} = \frac{D_r}{T} \times 100$ Where: R_{reading} = successful ratio of reading calculation, D_r = Number of reading calculations, T = total dataset size	43.75%
Unit of Measure Detection	$R_{\text{UoM}} = \frac{D_{\text{UoM}}}{T} \times 100$ Where: R_{UoM} = successful ratio of unit of measure detection, D_{UoM} = Number of unit of measure detections, T = total dataset size	41.25%

The reasons of table [6.0] results for pointer tip detection, reading calculation and unit of measure detection are varies, one of the most common reasons is the shadow that created beneath gauge protection case and the area above the scale gradin where the pointer tip is located:

**Figure 29.** Shadow effect on pointer tip detection

Other reasons are related to gauge's glass condition and image capture perspective; however, the overall performance result calculation is:

Table 14. Model Overall Results (Table 7.0)

Feature	Gauge Detection	Pointer Base Detection	Pointer Tip Detection (R_{tip})	Reading Calculation (R_{reading})	Unit of Measure Detection (R_{UoM})
Ratio	100%	100%	43.75%	43.75%	41.25%

$$\begin{aligned}
 N_{\text{model}} &= \text{Avg}(\text{Gauge Detection, Pointer Base Detection, } R_{\text{tip}}, R_{\text{reading}}, R_{\text{UoM}}) \\
 &= \frac{100 + 100 + 43.75 + 43.75 + 41.25}{5} \\
 &= 65.75\%
 \end{aligned} \tag{3}$$

3.2. Case Study, centrifugal compressor discharge pressure reading and warning:-

Another aspect of model deployment and evaluation is to test of the most important features of the model which is "estimate the process features and determine the safety barrier of the process system", but before moving their, we need to understand first the engineering process for designing and constructing the industrial facilities which is a sequential process that goes through several engineering and designing steps:

Table 15. Project Study Phases

Study Phase	Sequence
Conceptual Study	Seq 1
Pre-feasibility Study	Seq 2
Feasibility Study	Seq 3
Front End Engineering Design / FEED	Seq 4
Detailed Design	Seq 5
Project Scope of Work preparation	Seq 6

In all mentioned steps the facility details is being identified, calculated and therefore designed, more specifically, in sequence (4) -front End Engineering Design or commonly known as FEED- all facility's drawing shall be finalized with all require data regarding each individual equipment, instrument and skid, however, one of the most important diagrams or drawing is call it "PID" which stand for "piping and instrumentation diagrams", PIDs shows the piping and related components of a physical process flow and the operation boundaries for each equipment in the diagram. The idea here is to learn the model with initial operation boundary for each gauge distributed in the plant, PIDs will provide such information to the model as pre-trained data before deployment phase. To put things with its context for above mentioned explanation, we propose a case study for the centrifugal compression unit of Brega Compressor station, figure [27.0] shows the PID of this unit:

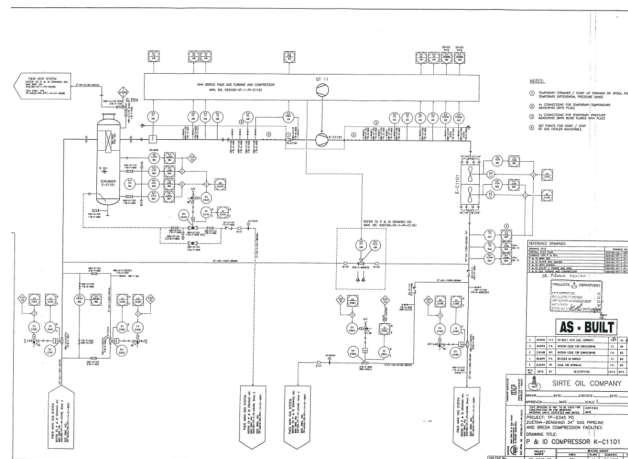


Figure 30. PI&D for compression unit

If we focus the discharge line of compressor we notice that the maximum outlet design pressure is (53 Barg = 768 psig) as figure [29.0] shows

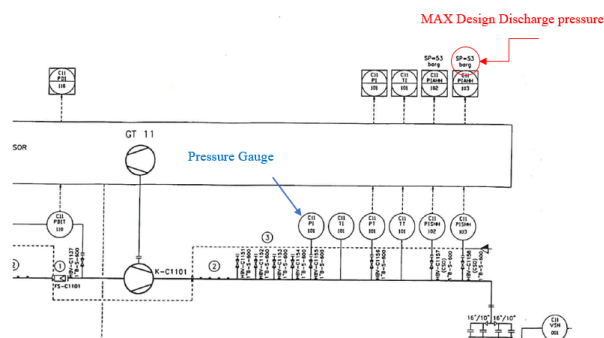


Figure 31. Compressor max design discharge pressure

On site and in this specific point, a pressure gauge has been installed that has a TAG number: "C11-PI-101" as illustrated in figure [29.0] PID, however, this information is vital that can we utilized it to tell the model where the location of the gauge and therefore the estimate the process boundaries:



Figure 32. C11-PI-101 pressure gauge

When we applied the model on this gauge the result will be:

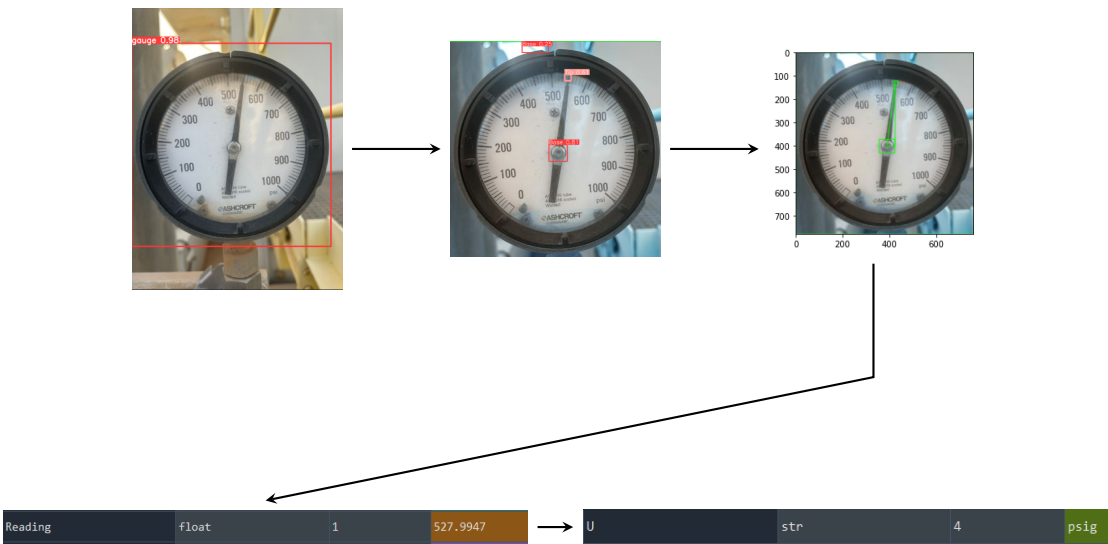


Figure 33. The model applying on C11-PI-101 pressure gauge

And by comparing the model results with input data from the P&ID for maximum operation condition in this point of measuring, the reading datasheet will be:

Company Logo		Company Name		Issued Date		Edit Date		Form No. XXX-XXXXXX Reading Sheet		Page 1 of 1	
Unit (XXXXX) Reading											
Date: XXXX/XX/XX	Time	00:00	10:00	13:00	14:00	15:00	16:00	17:00	18:00	End of Measure	Remarks
Feed Gas Inlet Meter Pressure	1317.2										Normal Operation
U-2 Discharge Pressure	1317.2										100% flow at the inlet & 100% flow at the discharge outlet
C-11 Discharge pressure	1317.2										Normal Operation
Output Reading											

Figure 34. Model output reading

3.3. Model Error Calculations:-

The final part of this section is to analysis the model readings discrepancy when we compare with actual readings as figure [33.0] shows:

Table 16. Comparison of Model Output vs. Actual Readings

Input Image	Actual Reading	Model Output
4.jpg	515	517.24
15.jpg	418	447.81
38.jpg	390	416.15
73.jpg	565	550
76.jpg	500	494
77.jpg	150	162.9

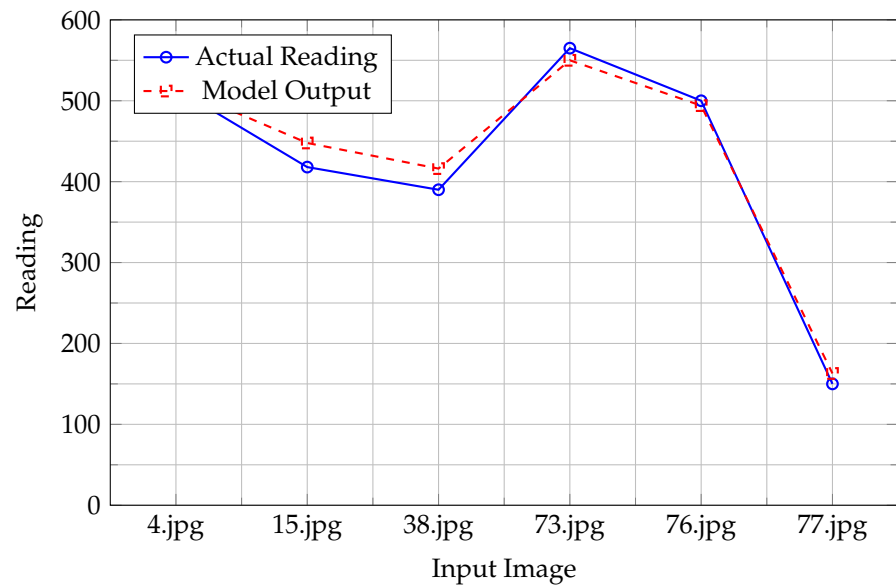


Figure 35. Comparison of Model Output vs Actual Reading

To calculate the model error, we can use the following equation:

$$\text{Model Error (\%)} = \frac{\text{Model Output}}{\text{Actual Reading}} \times 100 \quad (22.0)$$

Where:

- **Model Output** = The reading predicted by the model.
- **Actual Reading** = The true reading from the dataset.
- **Model Error (%)** = The percentage error of the model.

Therefor we can update table [8.0] to calculate overall error:

Table 17. Model Error and Deviation for Each Input Image

Input Image	Actual Reading	Model Output	Error (%)	Deviation
4.jpg	515	517.24	100.43	+0.43
15.jpg	418	447.81	107.13	+7.13
38.jpg	390	416.15	106.7	+6.7
73.jpg	565	550	97.34	-2.66
76.jpg	500	494	98.8	-1.2
77.jpg	150	162.9	108.6	+8.6

The term **Dev.** in Table 17 stands for the **deviation** between the model output reading and the actual reading. It is calculated using the following equation:

$$\text{Dev.} = |\text{Model Error (\%)} - 100| \quad (23.0)$$

4. Conclusions

In conclusion, we tested the model in several aspects starting from applied in newly 80 images that has NOT detected before, then we conducted a case study on centrifugal compressor discharge line pressure gauge to ensure its accuracy, lastly we calculate the discrepancy and the error of model output reading when we compare it the actual one.

References

1. Milana, E.; Ramírez-Agudelo, O.H.; Schmiedt, J.E. Autonomous Reading of Gauges in Unstructured Environments. *Sensors* **2022**, *22*, 6681. Available online: <https://www.mdpi.com/1424-8220/22/17/6681> (accessed on 3 September 2022).
2. Li, Z.; Zhou, Y.; Sheng, Q.; Chen, K.; Huang, J. A High-Robust Automatic Reading Algorithm of Pointer Meters Based on Text Detection. *Sensors* **2020**, *20*, 5946. Available online: <https://www.mdpi.com/1424-8220/20/20/5946> (accessed on 21 October 2020).
3. Li, Y.; Li, X.; Zhang, C. Pointer Meter Reading Recognition by Joint Detection and Segmentation. *Appl. Sci.* **2024**, *14*, 1443. Available online: <https://www.mdpi.com/2076-3417/14/4/1443> (accessed on 9 February 2024).
4. Gallagher, J. Turn Analog Dials into Digital Data with Computer Vision. Roboflow Blog. Available online: <https://blog.roboflow.com/read-analog-dials-computer-vision/> (accessed on 28 November 2022).
5. Leon-Alcazar, J.; Alnumay, Y.; Zheng, C.; Trigui, H.; Patel, S.; Ghanem, B. Learning to Read Analog Gauges from Synthetic Data. *arXiv* **2023**, arXiv:2308.14583. Available online: <https://arxiv.org/abs/2308.14583> (accessed on 28 August 2023).
6. Peixoto, J.; Sousa, J.; Carvalho, R.; Santos, G.; Cardoso, R.; Reis, A. End-to-End Solution for Analog Gauge Monitoring Using Computer Vision in an IoT Platform. *Sensors* **2023**, *23*, 9858. Available online: <https://www.mdpi.com/1424-8220/23/24/9858> (accessed on 15 December 2023).
7. Tran, M.Q.; Tran, H.T.; Pham, V.C. Computer Vision System for Reading Analog Gauges at Power Substation. In Proceedings of the *International Symposium on Electrical and Electronics Engineering (ISEE)*, Ho Chi Minh City, Vietnam, 15–16 April 2021; pp. 147–152. IEEE. Available online: <https://ieeexplore.ieee.org/document/9418754> (accessed on 15 April 2021). doi:10.1109/ISEE51682.2021.9418754.
8. Samnejad, M.; Aboelatta, M.; Dung, C.V. Asset Inspection Powered by Computer Vision: The Use of Deep Neural Networks for Automating the Detection and Classification of Pipeline External Damage. Pipeline Research Council International, Inc., 2021. Available online: <https://www.academia.edu/48971476> (accessed on 2021). PRCI-REX2021-040.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.